# Introduction to Deep learning

Ngô Minh Nhựt

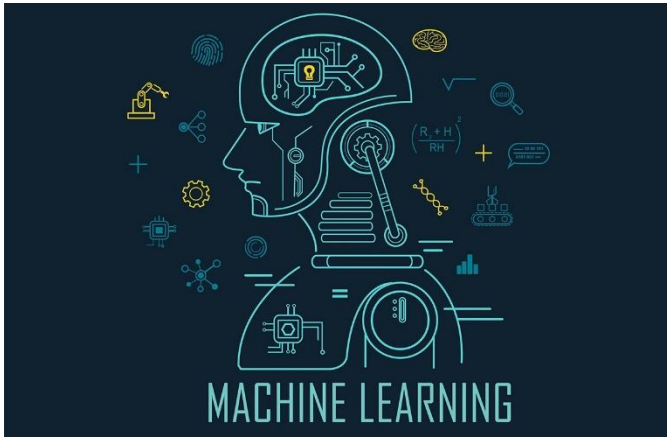
2024

# Outline

- **Introduction**
- Neural network
- Convolutional neural network
- Practice with Google Colab

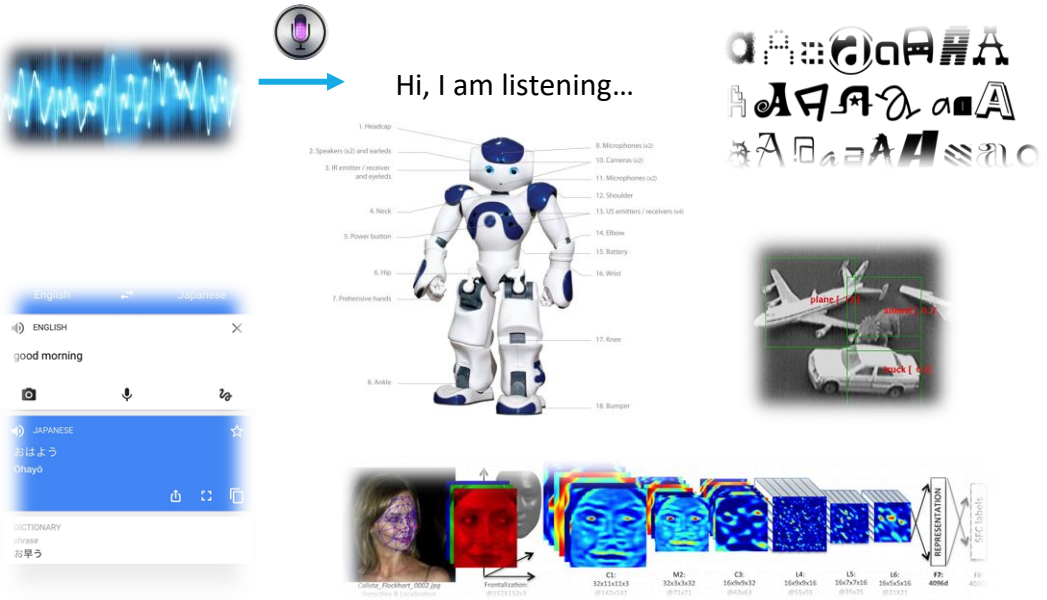
# Machine learning

- A machine learning algorithm is an algorithm learning to accomplish a task by observing data.
  - Used on complex tasks where it's hard to develop algorithms with handcrafted-rules
  - Exploits patterns in observed data and extract rules automatically



# Applications

- Computer vision
- Speech to text, text to speech
- Financial analysis
- Self-driving cars
- Ads-targeting
- Virtual assistance



# Example: object detection

- Learn from annotated corpus of examples (a dataset) to classify unknown images among different object types
- Observe images to learn patterns
- Lot of data available (i.e: ImageNet dataset)
- Very good error rates (< 5% with deep-CNN)

# Types of machine learning algorithms

## Supervised

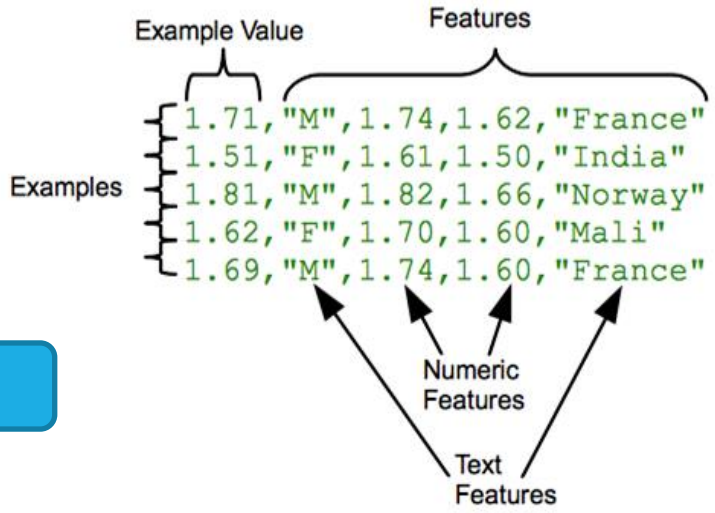
- Learn a function by observing labeled samples, containing input and expected output
- Classification
- Regression

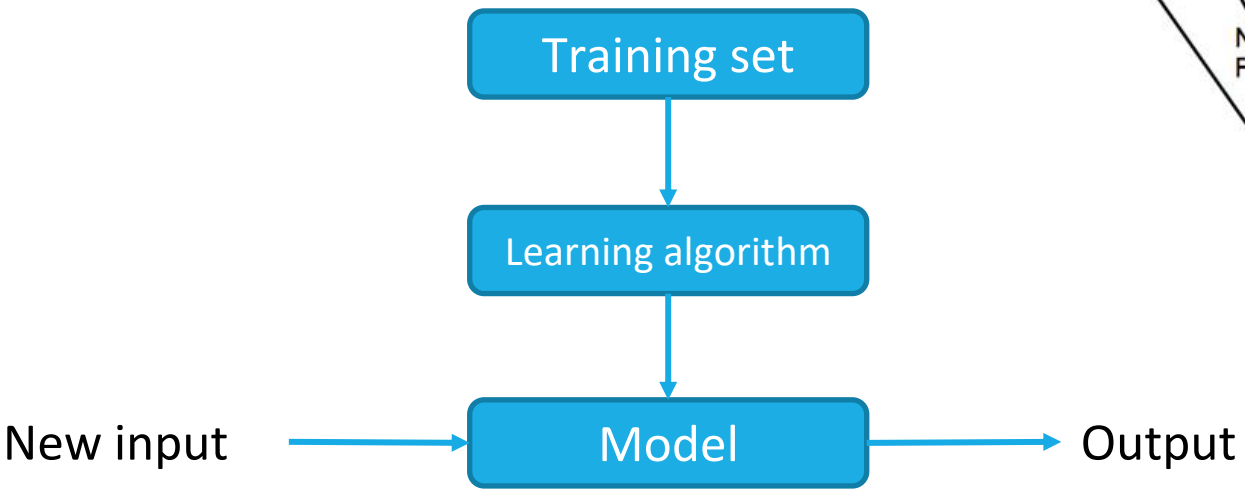
## Unsupervised

- Find underlying relations in data by observing unlabeled samples, containing only input
- Clustering
- Dimensionality reduction

# Machine learning

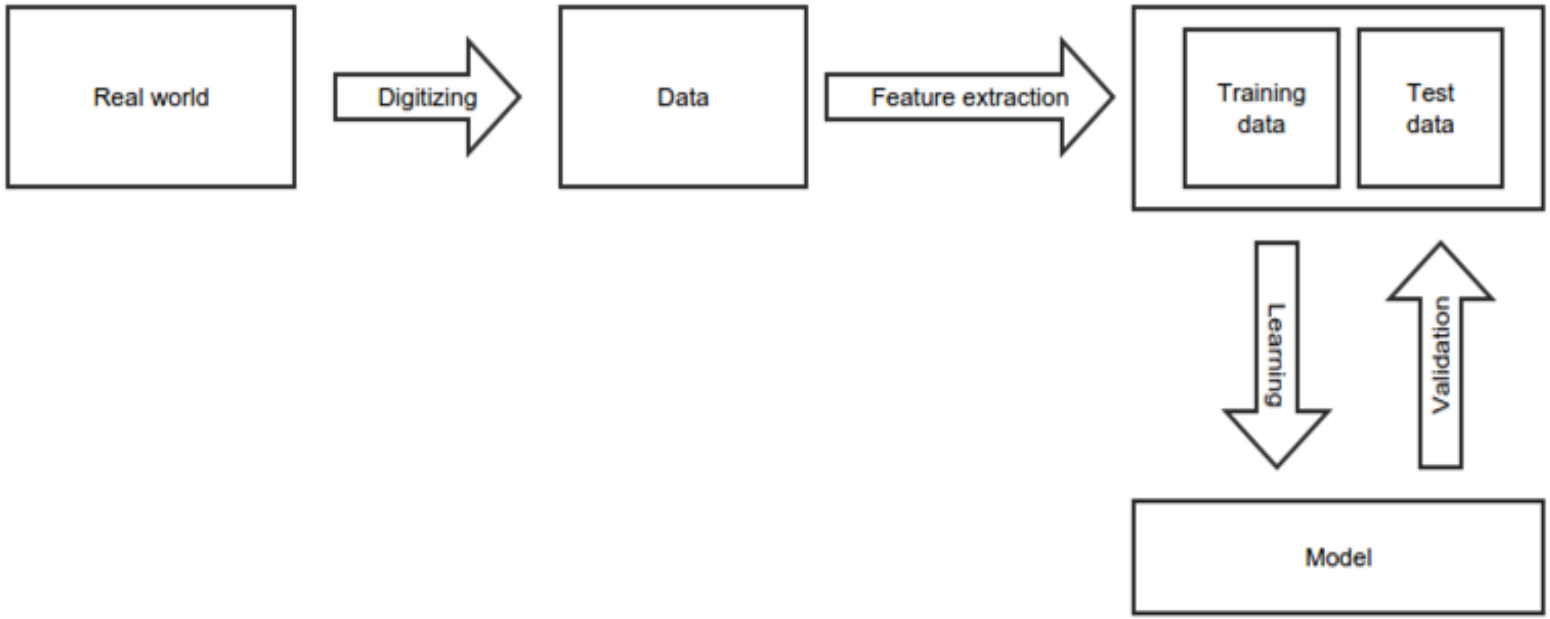
- Training set



# Machine learning

- Learning process



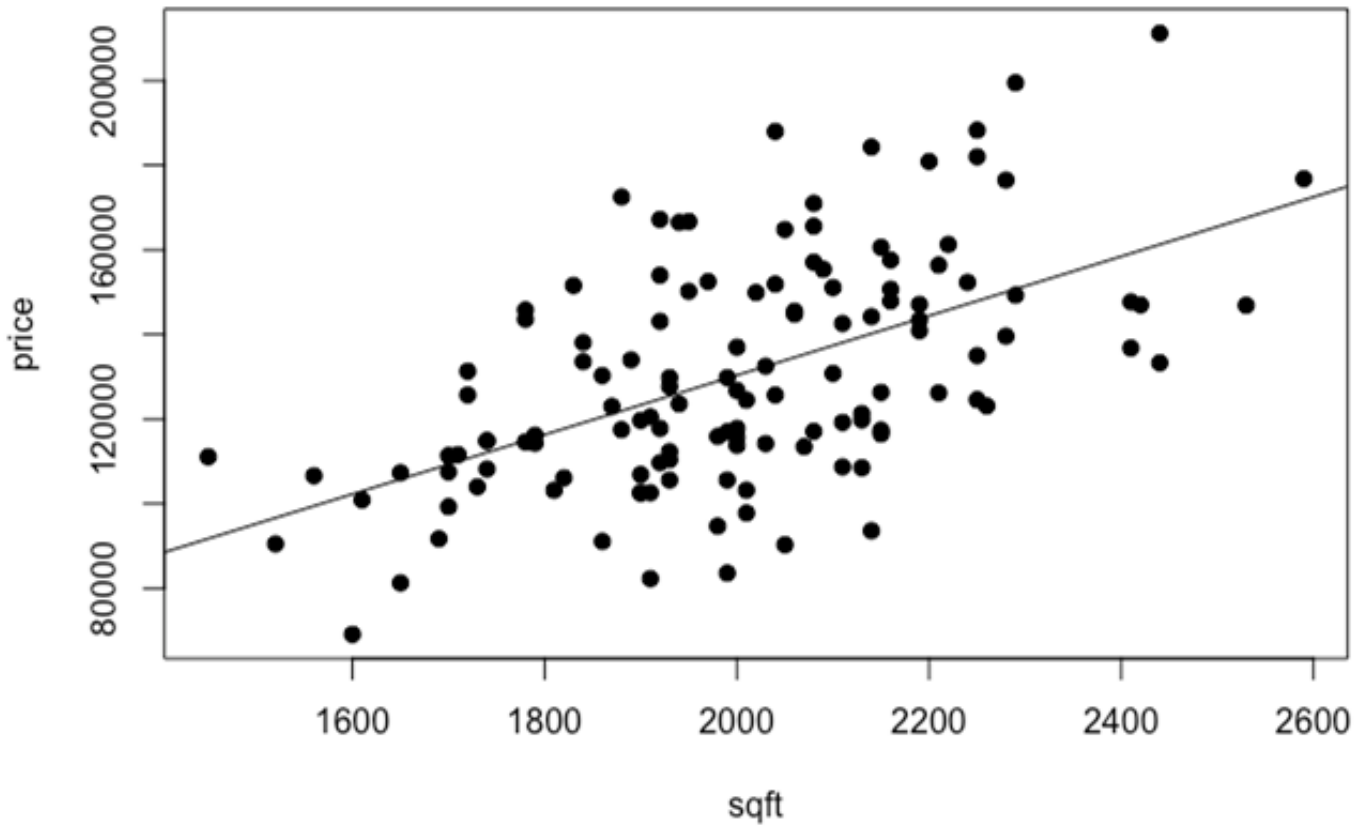
# Feature extraction

- Extract the information useful for the task (features) we want to learn
- Example
  - Stock market time-serie → [opening price, closing price, lowest, highest]
  - Image → Image with edges filtered
  - Document → bag-of-word

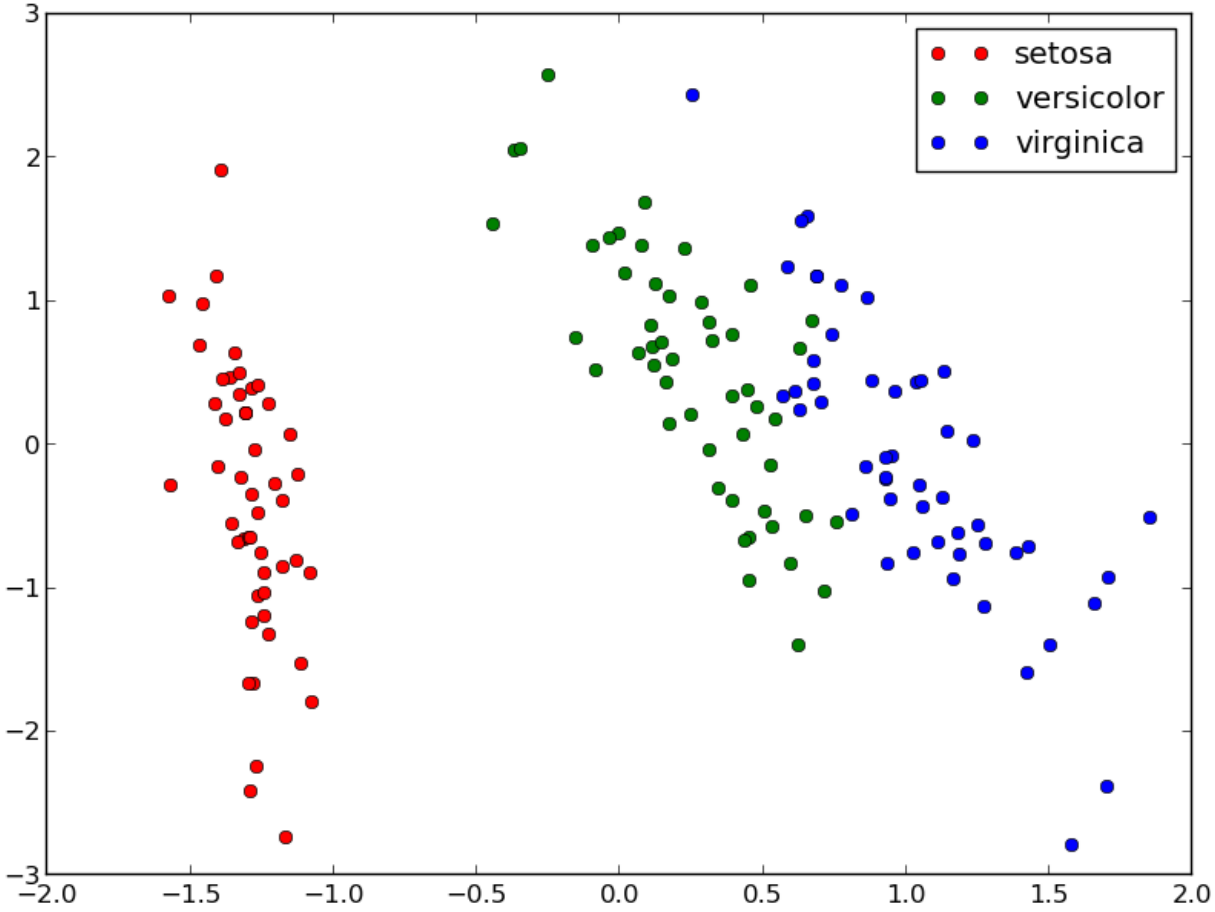
# Classification vs Regression

- Regression: learn a function mapping an input to a real value
  - E.g., predict the temperature of tomorrow given some meteo signals
- Classification: learn a function mapping an input element to a class (within a finite set of possible classes)
  - E.g., predict tomorrow weather: {sunny, cloudy, rainy} given some meteo signals

# Regression

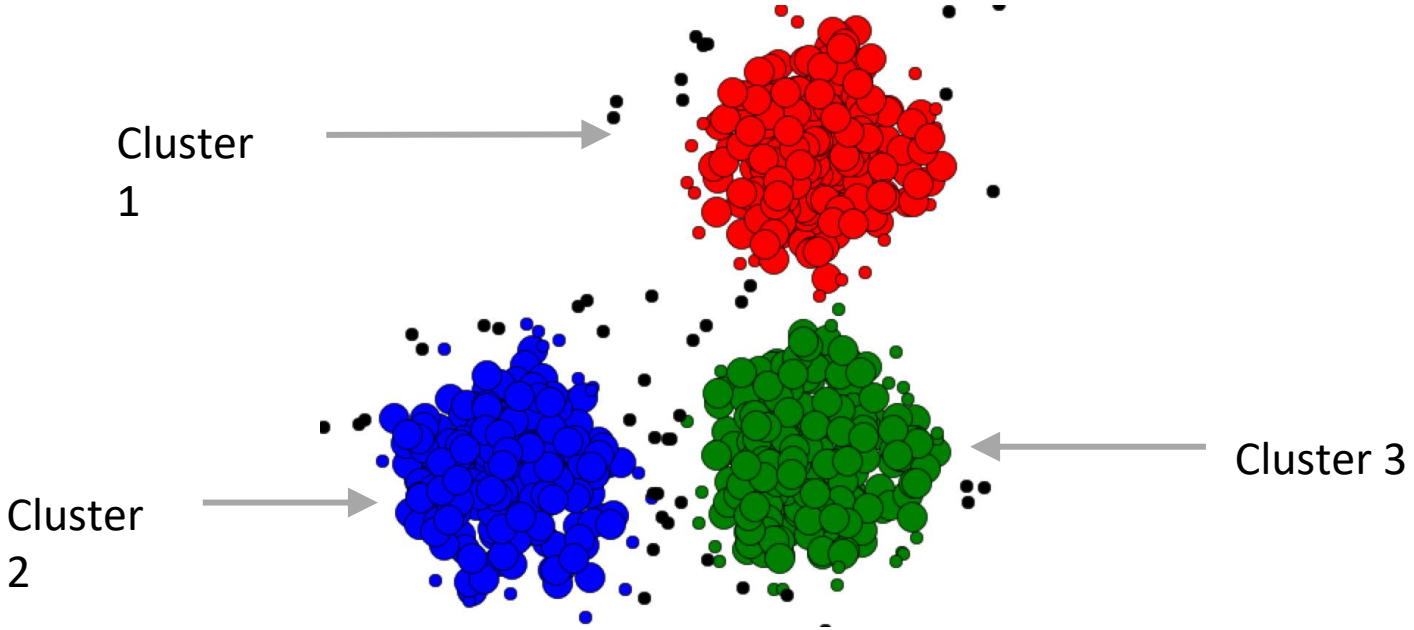

price
80000
120000
160000
200000
1600
1800
2000
2200
2400
2600
sqft

# Classification



# Clustering

- Separate different observed data points in similar groups (clusters). Observered data are unlabeled.

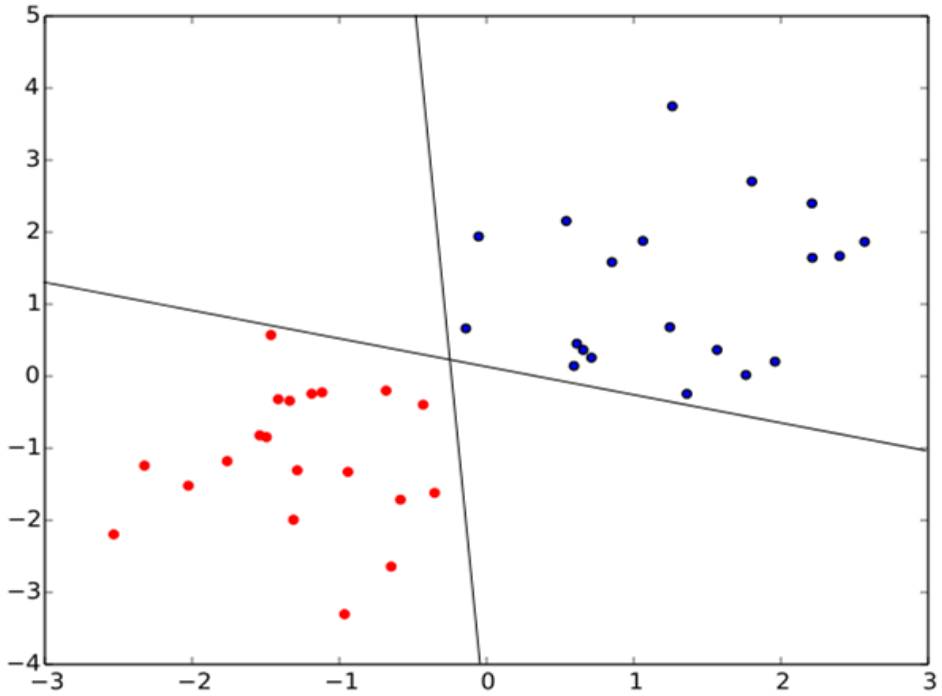


# Outline

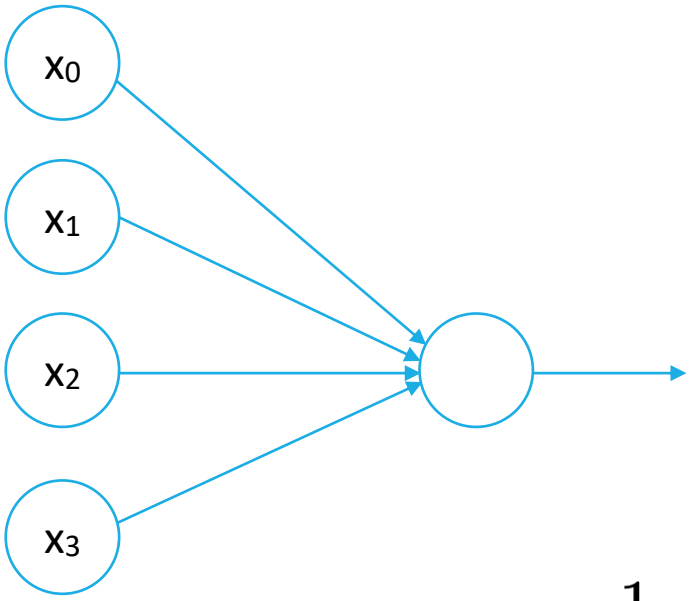
- Introduction
- **Neural network**
- Convolutional neural network
- Practice with Google Colab

# Logistic regression: binary classifier

- Learn a hyperplane to separate two class of data-points



# Logistic regression: binary classifier

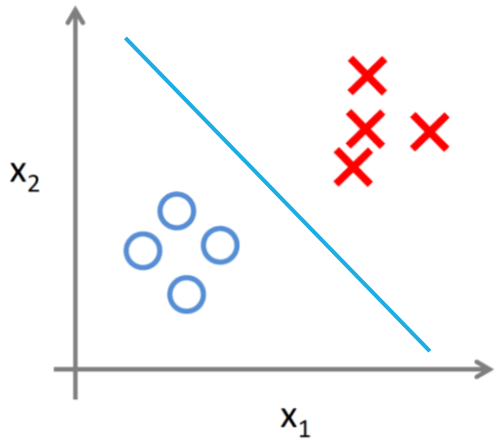



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

$h_\theta(x)$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Sigmoid function



# Logistic regression: loss function

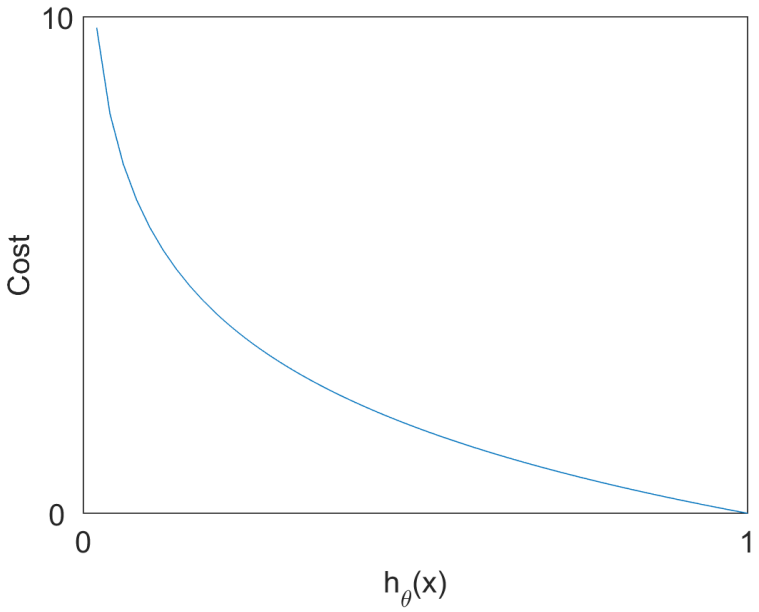
- Cross-entropy loss

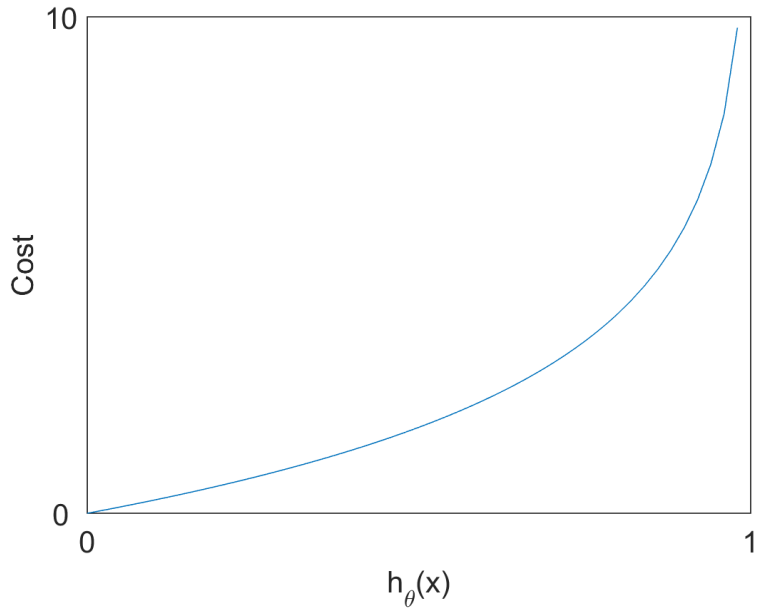
$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log h\left(x^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - h\left(x^{(i)}\right)\right)$$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

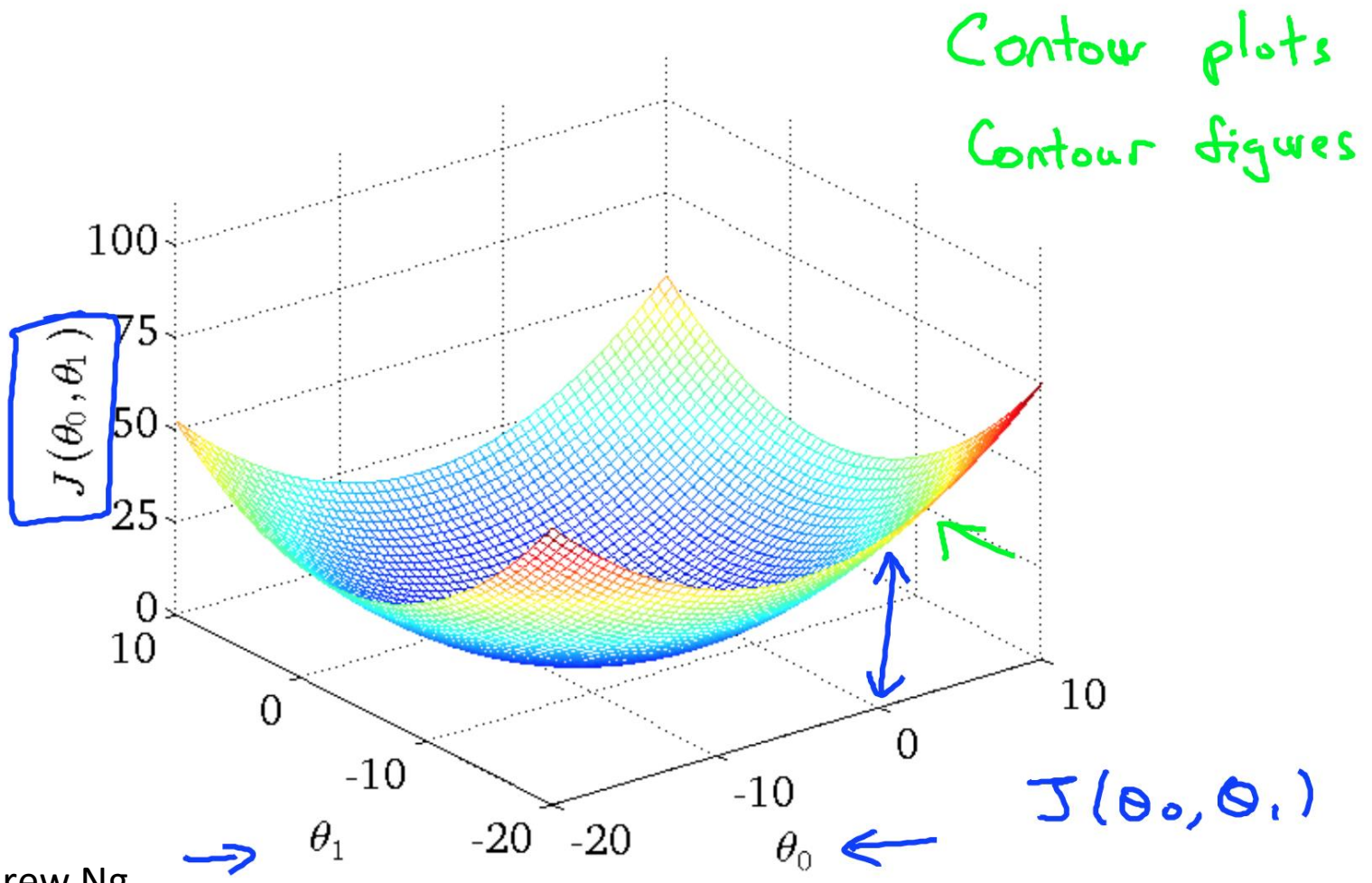
# Logistic regression: loss function

$$\text{Cost}(h(x), y) = \begin{cases} -\log h_\theta(x) & \text{if } y = 1 \\ -\log\big(1 - h_\theta(x)\big) & \text{if } y = 0 \end{cases}$$



# Logistic regression: loss function




Source: Andrew Ng

# Gradient descent algorithm

- Find a set of parameters where loss function is minimal
  - Loss function

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log h\left(x^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - h\left(x^{(i)}\right)\right)$$

  - Goal: search for a vector θ that makes J minimal

# Gradient descent algorithm

- Find a set of parameters where loss function is minimal
  - Loss function

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log h\left(x^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - h\left(x^{(i)}\right)\right)$$

  - Goal: search for a vector θ that makes J minimal
  - Strategy
    - Start from an initial point of θ
    - Change θ until J is minimal
  - Question: how to change?

# Gradient descent algorithm

- Partial derivative and gradient vector
  - $\frac{dJ}{d\theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$
  - $j = 0, 1, 2, \ldots, n$

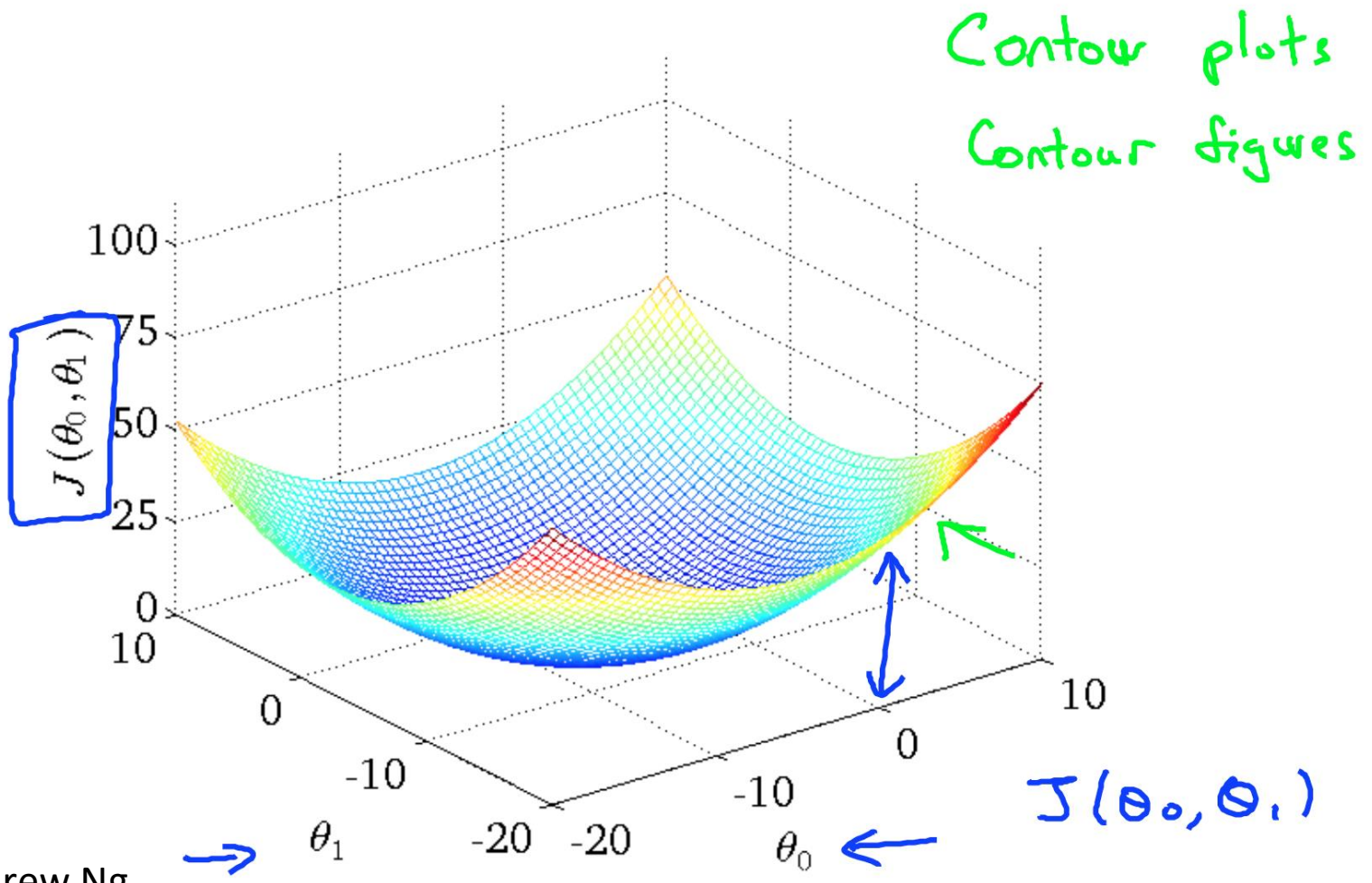
# Gradient descent algorithm

- Partial derivative and gradient vector
  - $\frac{dJ}{d\theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$
  - $j = 0, 1, 2, \dots, n$
- Vector gradient
  - $\left(\frac{dJ}{d\theta_0}, \frac{dJ}{d\theta_1}, \dots \frac{dJ}{d\theta_n}\right)$

# Gradient descent algorithm

- Partial derivative and gradient vector
  - $\frac{dJ}{d\theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$
  - $j = 0, 1, 2, \dots, n$
- Gradient vector
  - $\left(\frac{dJ}{d\theta_0}, \frac{dJ}{d\theta_1}, \dots \frac{dJ}{d\theta_n}\right)$
- Going to inverse direction of gradient vector makes loss function decrease

# Gradient descent algorithm




Source: Andrew Ng

# Gradient descent algorithm

Loop until convergence

{

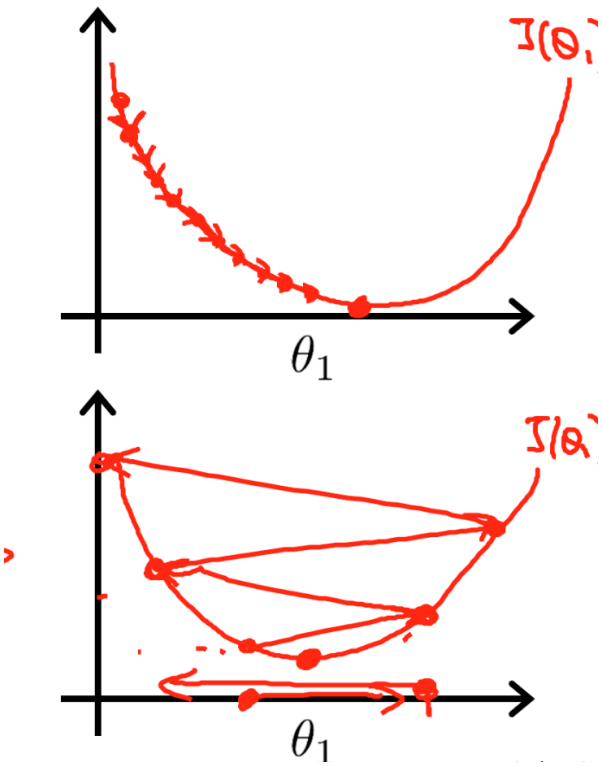
$$\theta_j = \theta_j - \alpha \frac{dJ}{d\theta_j}$$

}

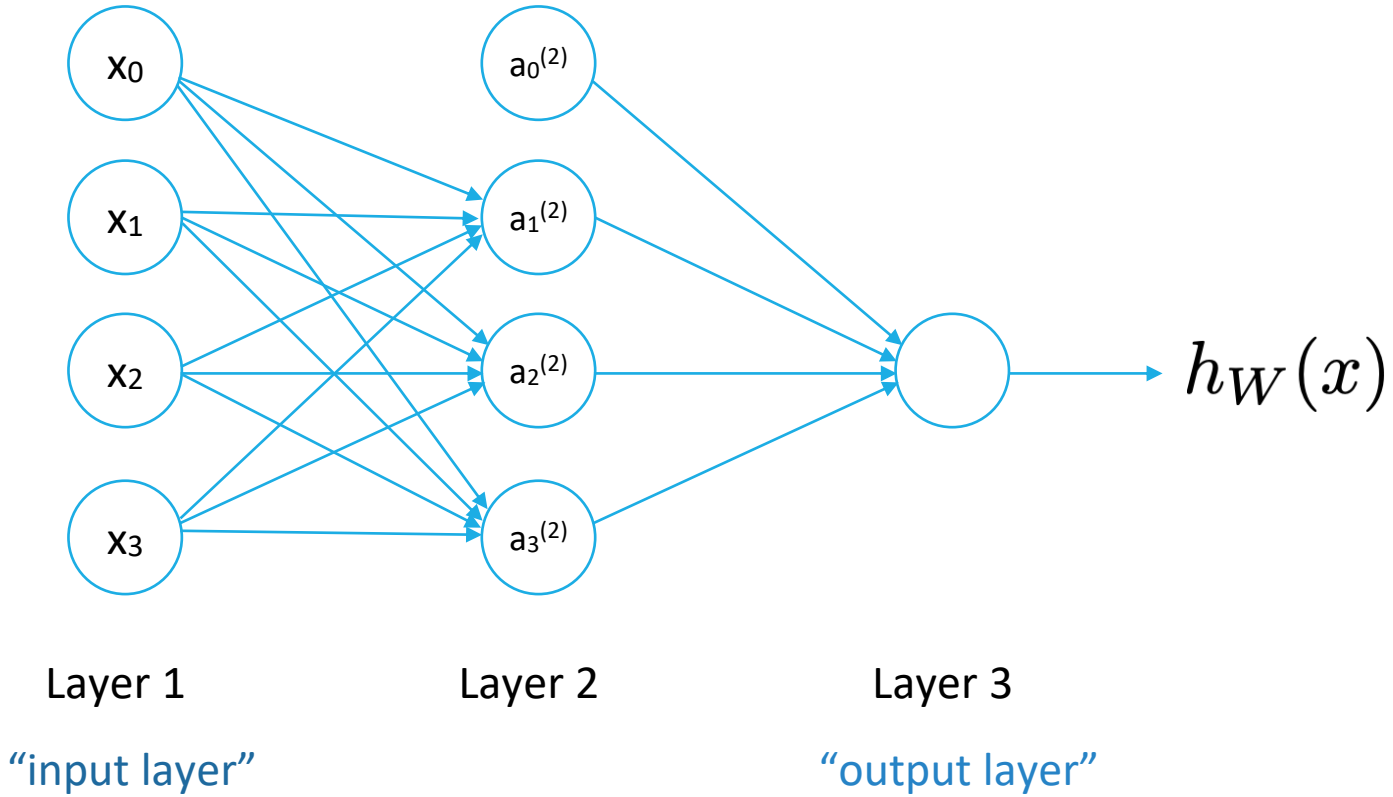
- α is learning rate
- Steps
  - Calculate gradient vector
  - Update elements of θ
  - Calculate loss

# Learning rate

- If $\alpha$ is too small, algorithm converges slowly
- If $\alpha$ is too big, algorithm may not converge

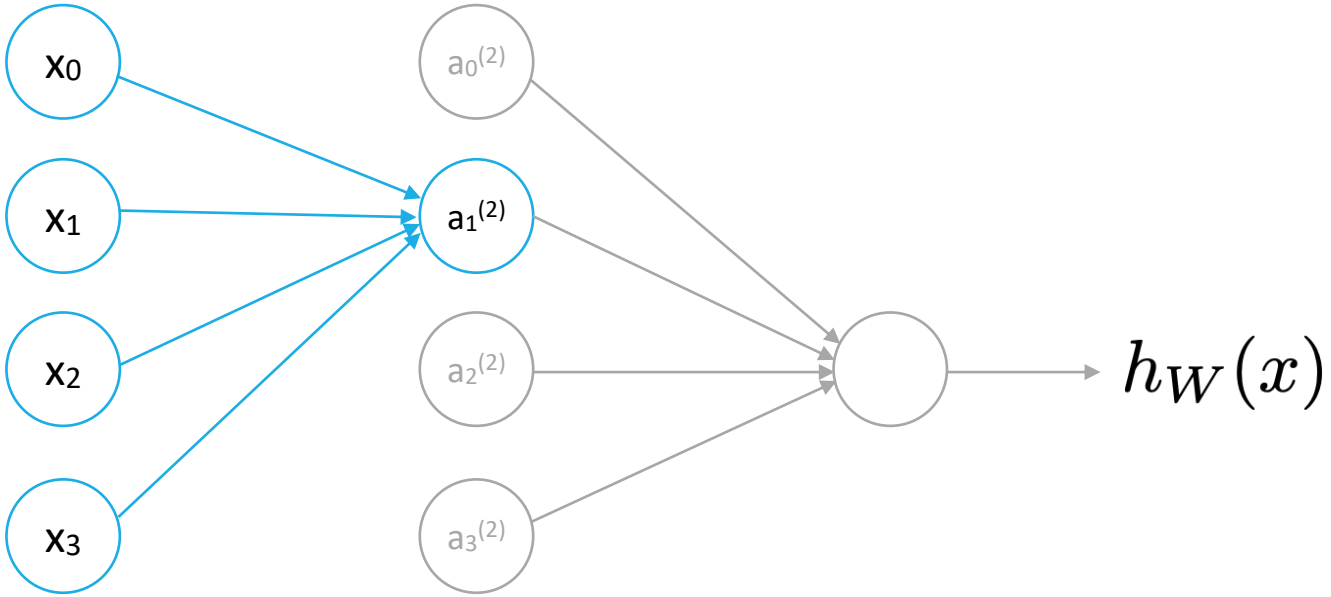


# Neural network



# Activation function

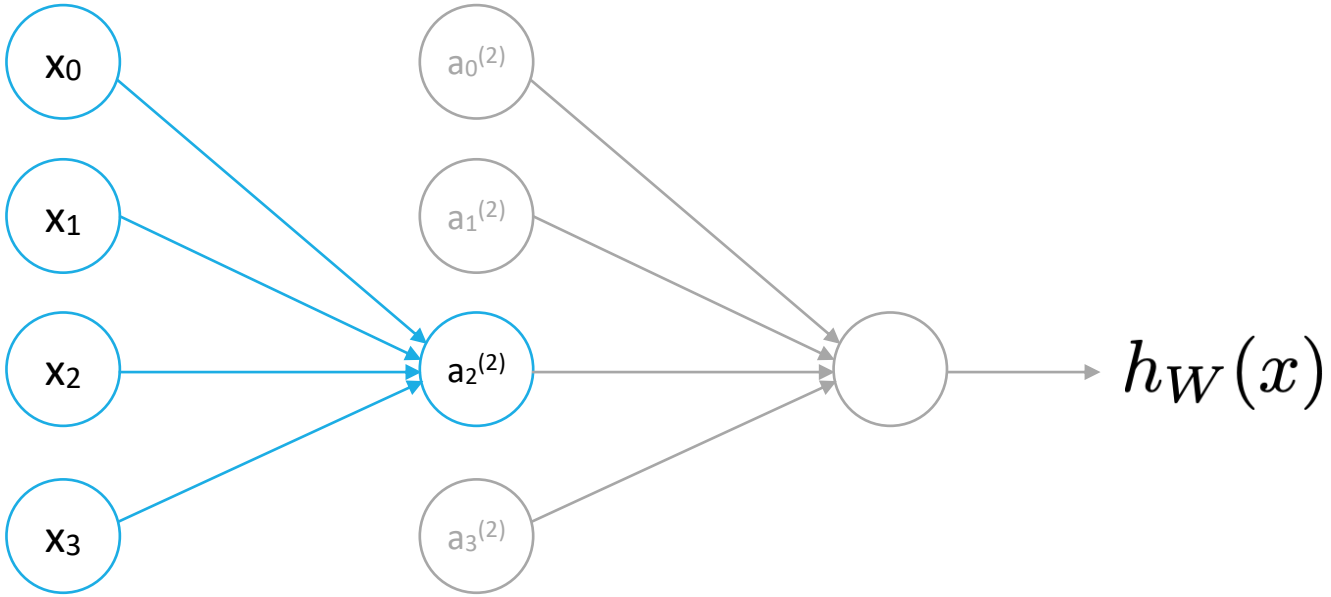
$$g(z) = \frac{1}{1 + e^{-z}}$$




$$a_1^{(2)} = g\left(W_{10}^{(1)}x_0 + W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3\right)$$

# Activation function

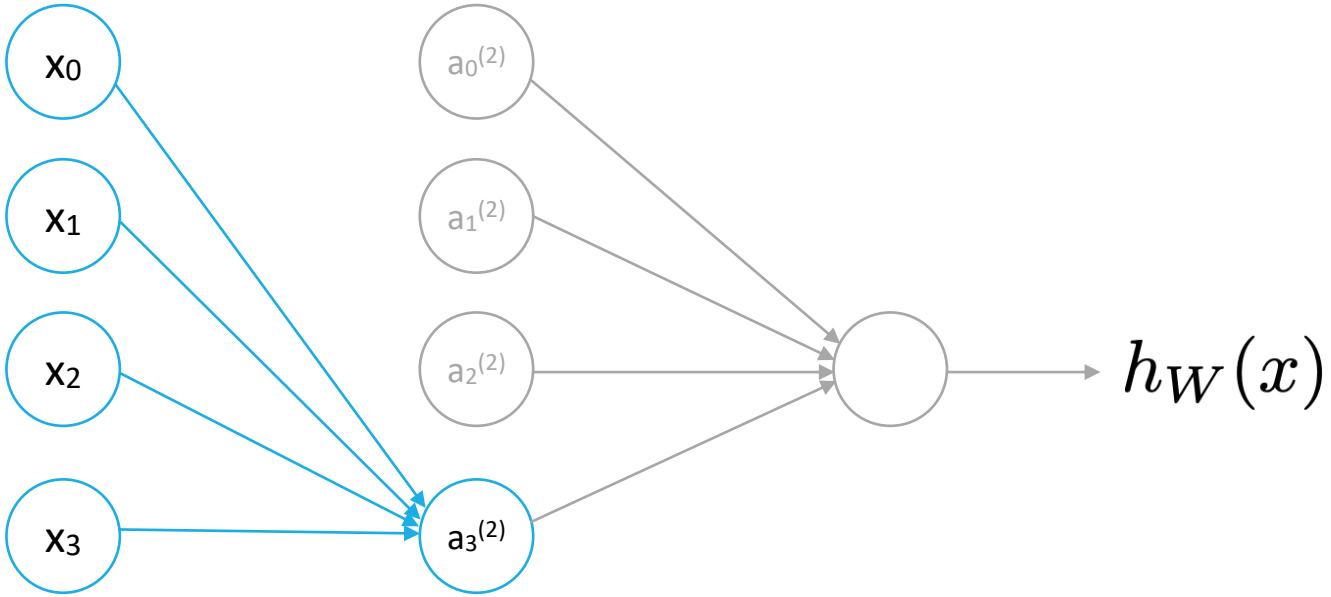
$$g(z) = \frac{1}{1 + e^{-z}}$$




$$a_2^{(2)} = g\left(W_{20}^{(1)}x_0 + W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3\right)$$

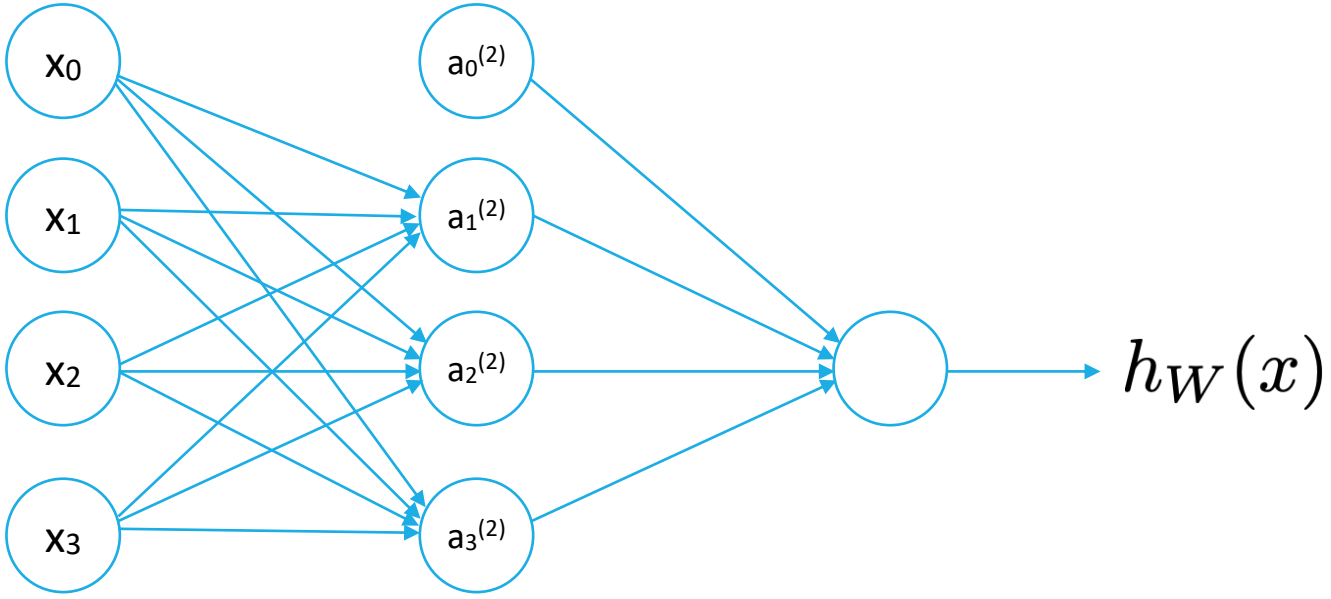
# Activation function

$$g(z) = \frac{1}{1 + e^{-z}}$$




$$a_3^{(2)} = g\left(W_{30}^{(1)}x_0 + W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3\right)$$

# Weight matrix

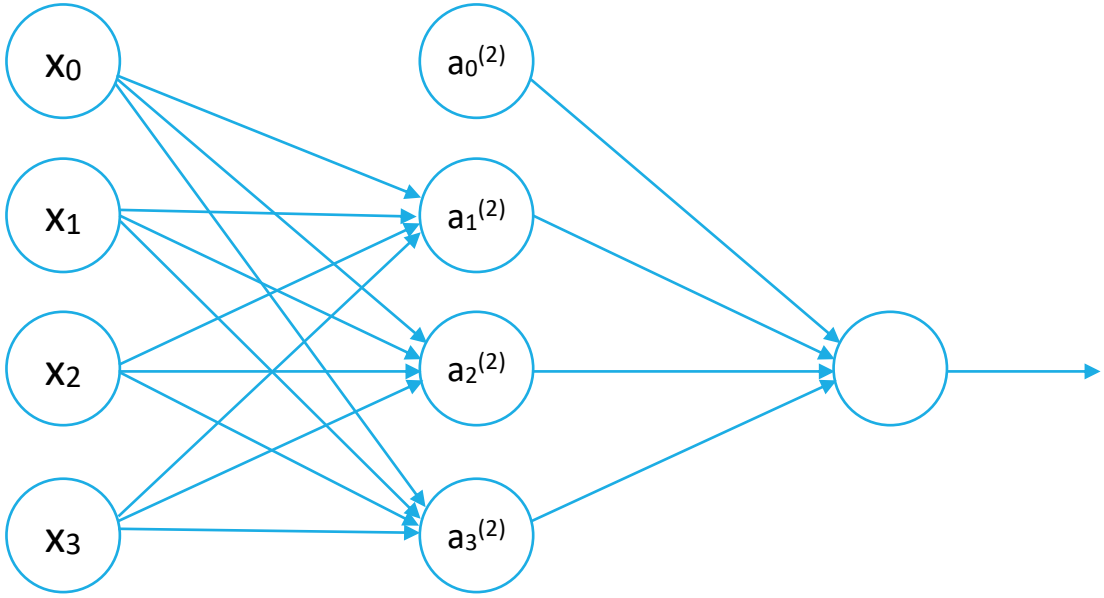



$a_i^{(j)}$: value (activation) of node i in layer j

$W^{(j)}$: weight matrix mapping values from layer j to layer j+1

If neural network has $s_j$ node in layer j and $s_{j+1}$ node in layer j+1, $W^{(j)}$ has a size of $s_{j+1}$ x ($s_j$ + 1)

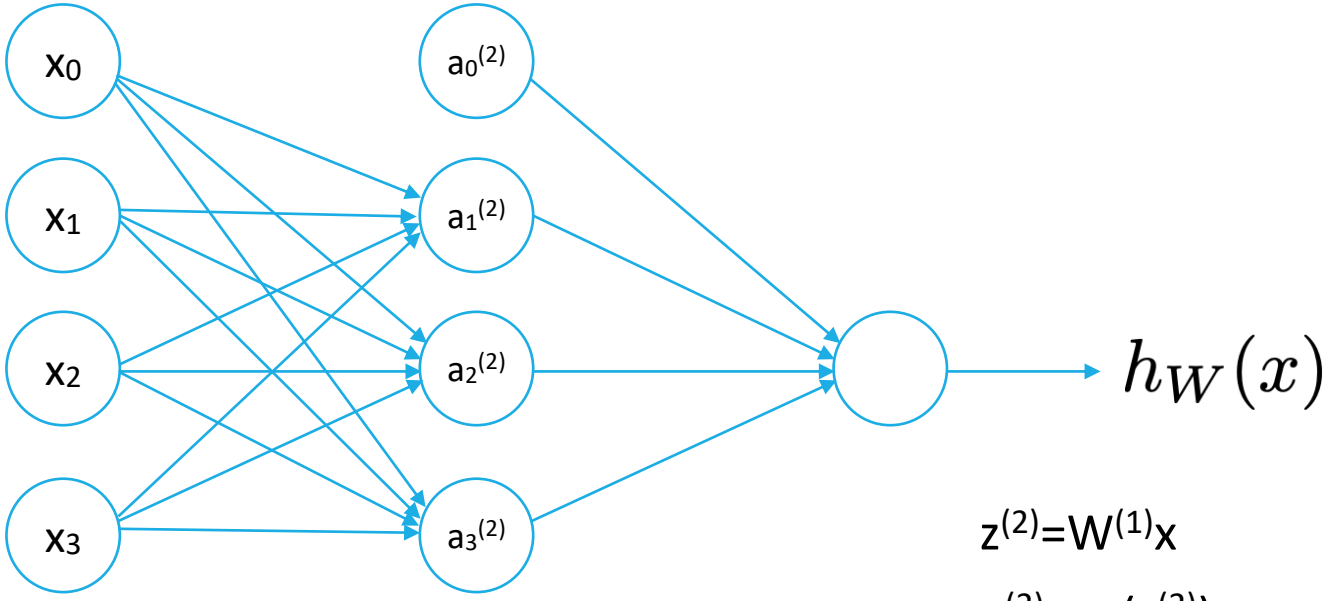
# Feedforward propagation




$$a_1^{(2)} = g\left(W_{10}^{(1)}x_0 + W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3\right)$$
$$a_2^{(2)} = g\left(W_{20}^{(1)}x_0 + W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3\right)$$
$$a_3^{(2)} = g\left(W_{30}^{(1)}x_0 + W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3\right)$$

$$h_W(x) = a_1^{(3)} = g\left(W_{10}^{(2)}a_0^{(2)} + W_{11}^{(2)}a_1^{(2)} + W_{12}^{(2)}a_2^{(2)} + W_{13}^{(2)}a_3^{(2)}\right)$$

# Vector representation




$$a_1^{(2)} = g\left(W_{10}^{(1)}x_0 + W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3\right)$$

$$a_2^{(2)} = g\left(W_{20}^{(1)}x_0 + W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3\right)$$

$$a_3^{(2)} = g\left(W_{30}^{(1)}x_0 + W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3\right)$$

$$h_W(x) = a_1^{(3)} = g\left(W_{10}^{(2)}a_0^{(2)} + W_{11}^{(2)}a_1^{(2)} + W_{12}^{(2)}a_2^{(2)} + W_{13}^{(2)}a_3^{(2)}\right)$$

$z^{(2)}=W^{(1)}x$

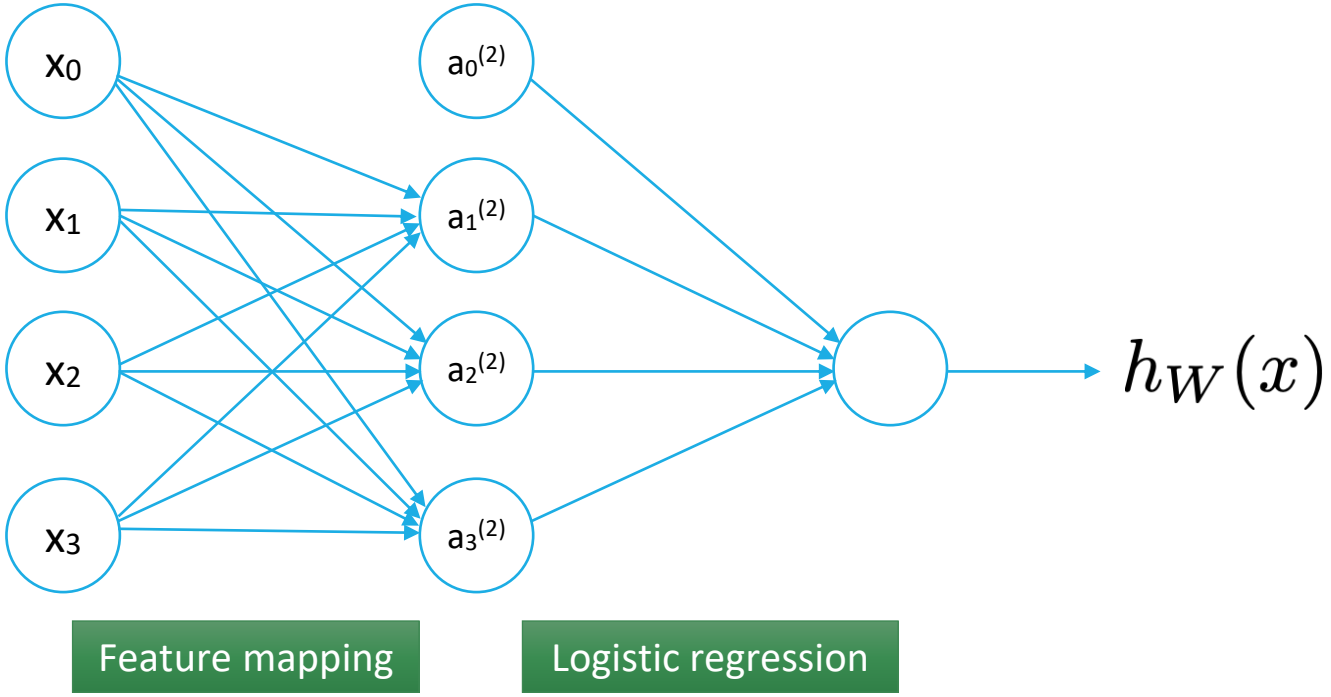
$a^{(2)} = g(z^{(2)})$

Thêm $a_0^{(2)} = 1$

$z^{(3)} = W^{(2)}a^{(2)}$

$h_W(x) = a^{(3)} = g(z^{(3)})$

# Feature self-learning network

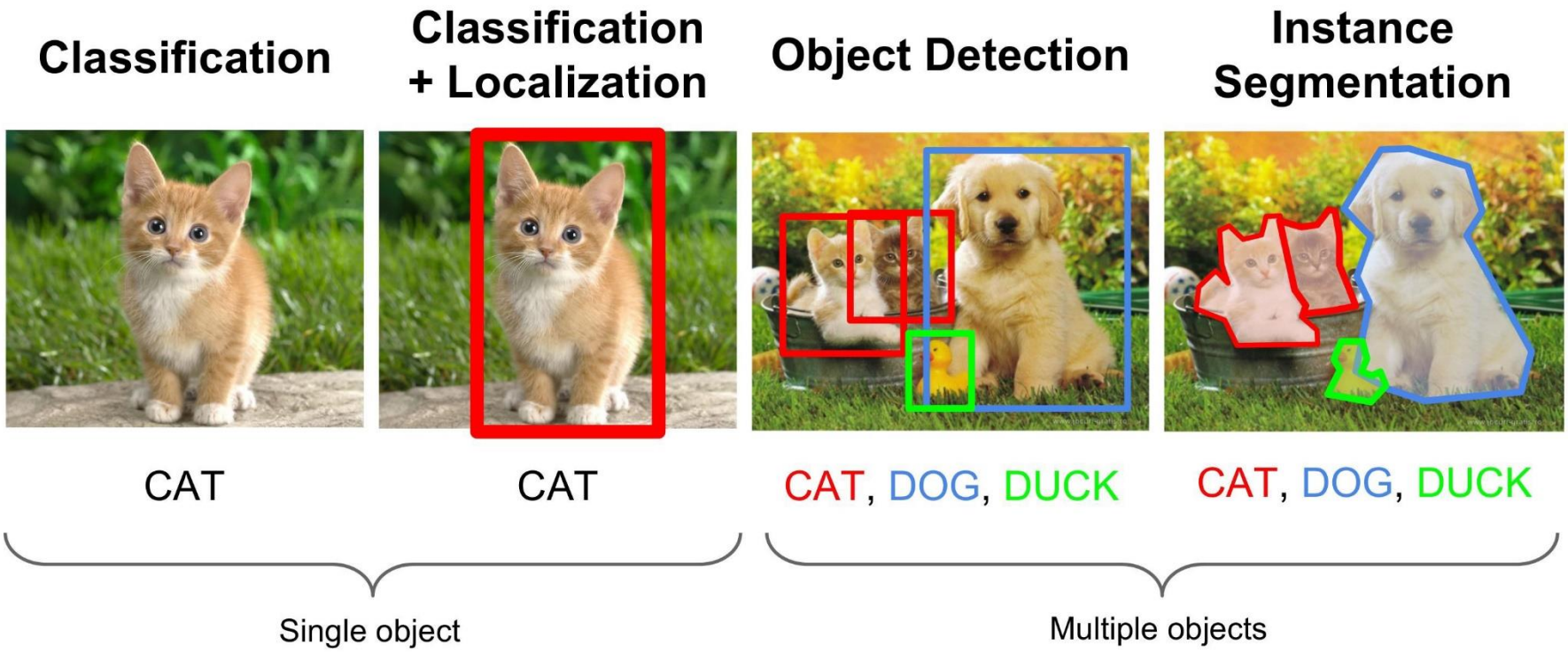



$$h_W(x) = a_1^{(3)} = g\left(W_{10}^{(2)} a_0^{(2)} + W_{11}^{(2)} a_1^{(2)} + W_{12}^{(2)} a_2^{(2)} + W_{13}^{(2)} a_3^{(2)}\right)$$

# Outline

- Introduction
- Neural network
- **Convolutional neural network**
- Practice with Google Colab

# Problems in computer vision



# Image segmentation


Semantic Segmentation


Instance Segmentation

# Segmentation techniques

- The pixel values will be different for the objects and the image's background → **Threshold Segmentation**
- **Edge Detection Segmentation -** There is always an edge between two adjacent regions with different grayscale values.
- **Image Segmentation based on Clustering**

# Convolutional neural network

- High resolution images contains O(millions) of pixels
- A neural network which can handle that kind of images would also have O(millions) of weight

**Convolutional filter**

# Convolutional neural network

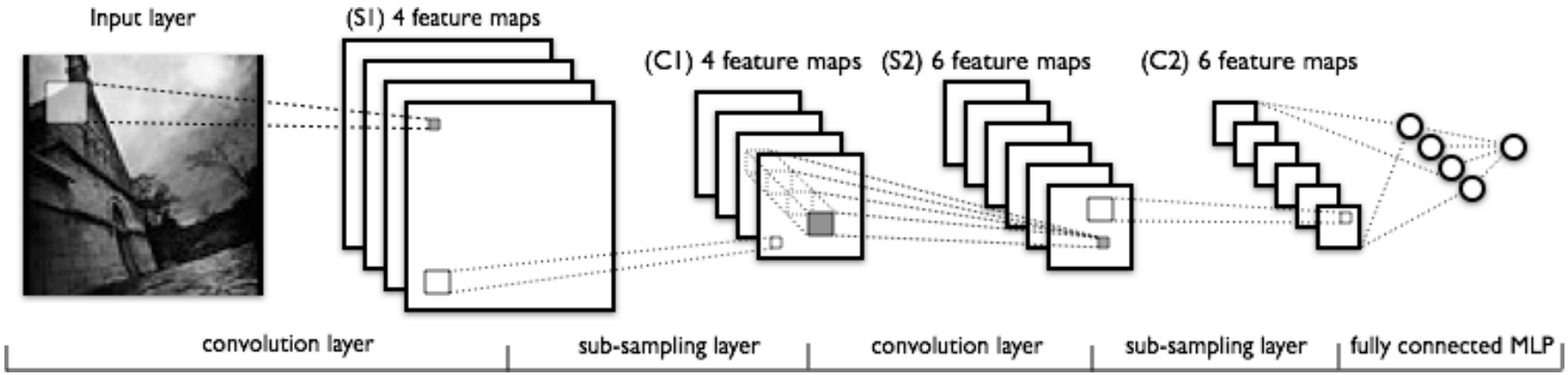
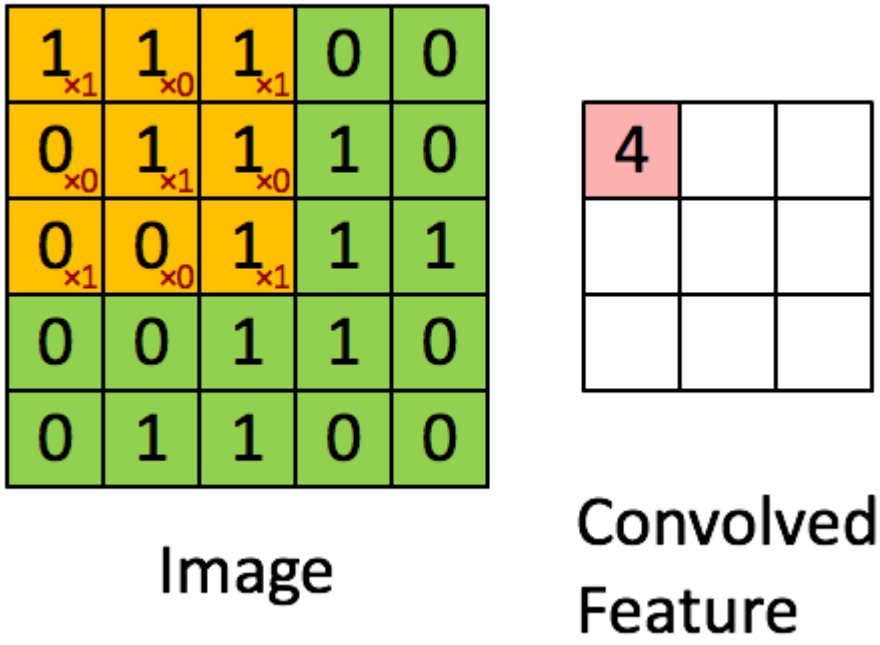



*Illustration: Theano documentation*

This kind of architecture will learn filters and build an internal representation of the input data using many stacked layers and finally use this representation on a classification task.

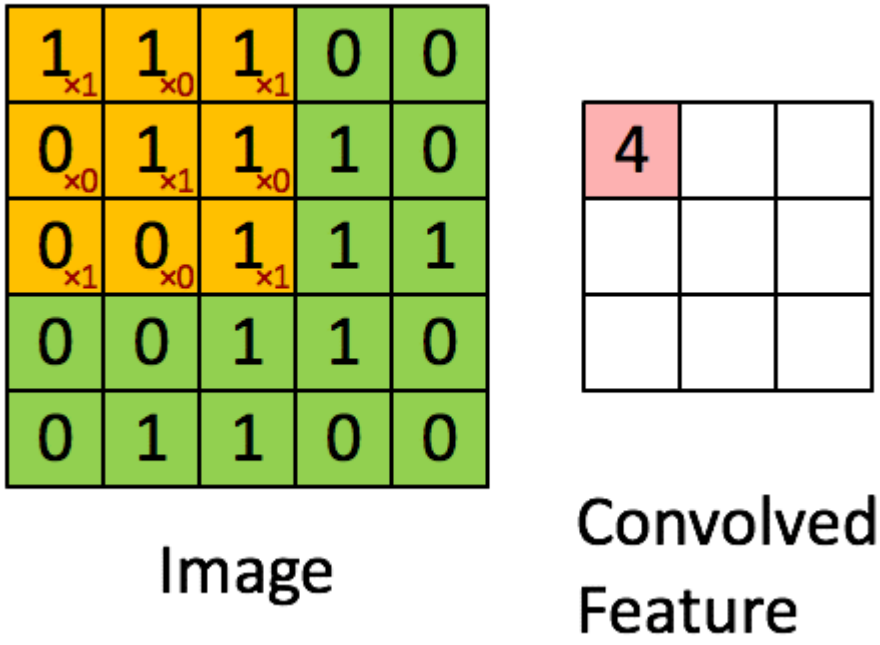
# Convolutional neural network

- Filters



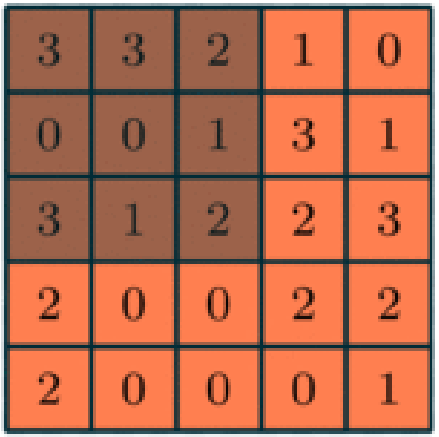
# Convolutional neural network

- Filters



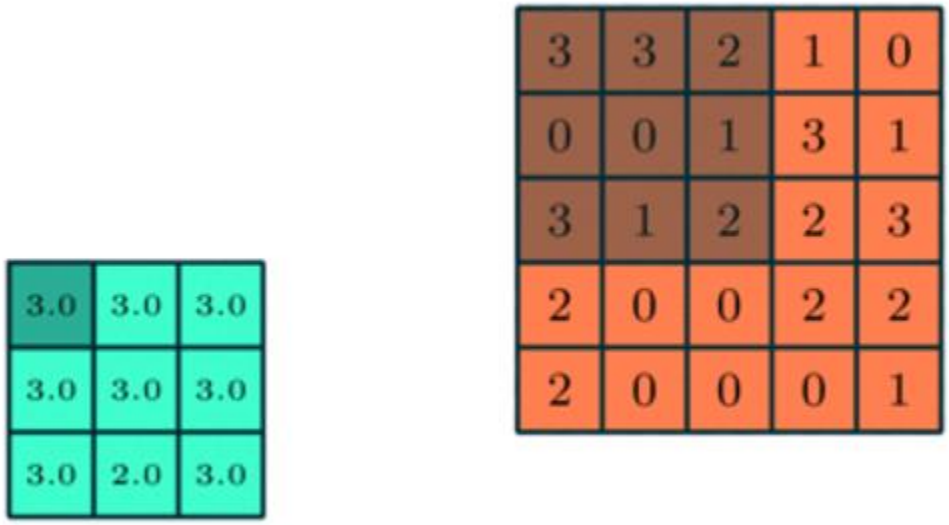
# Convolutional neural network

- Pooling

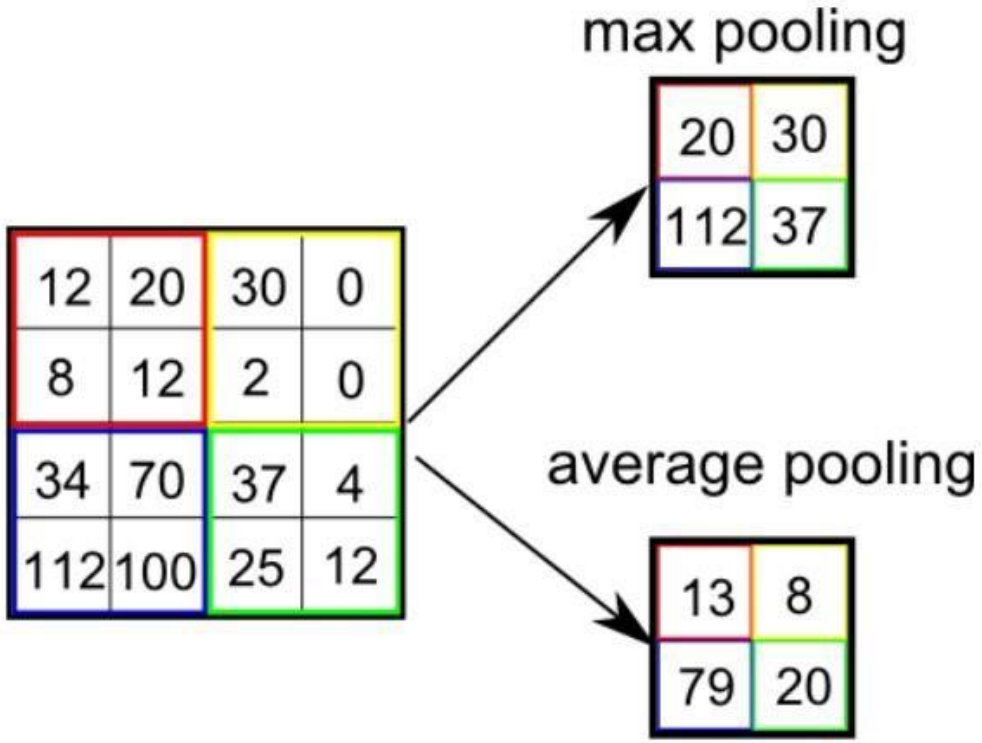
# Convolutional neural network

- Pooling

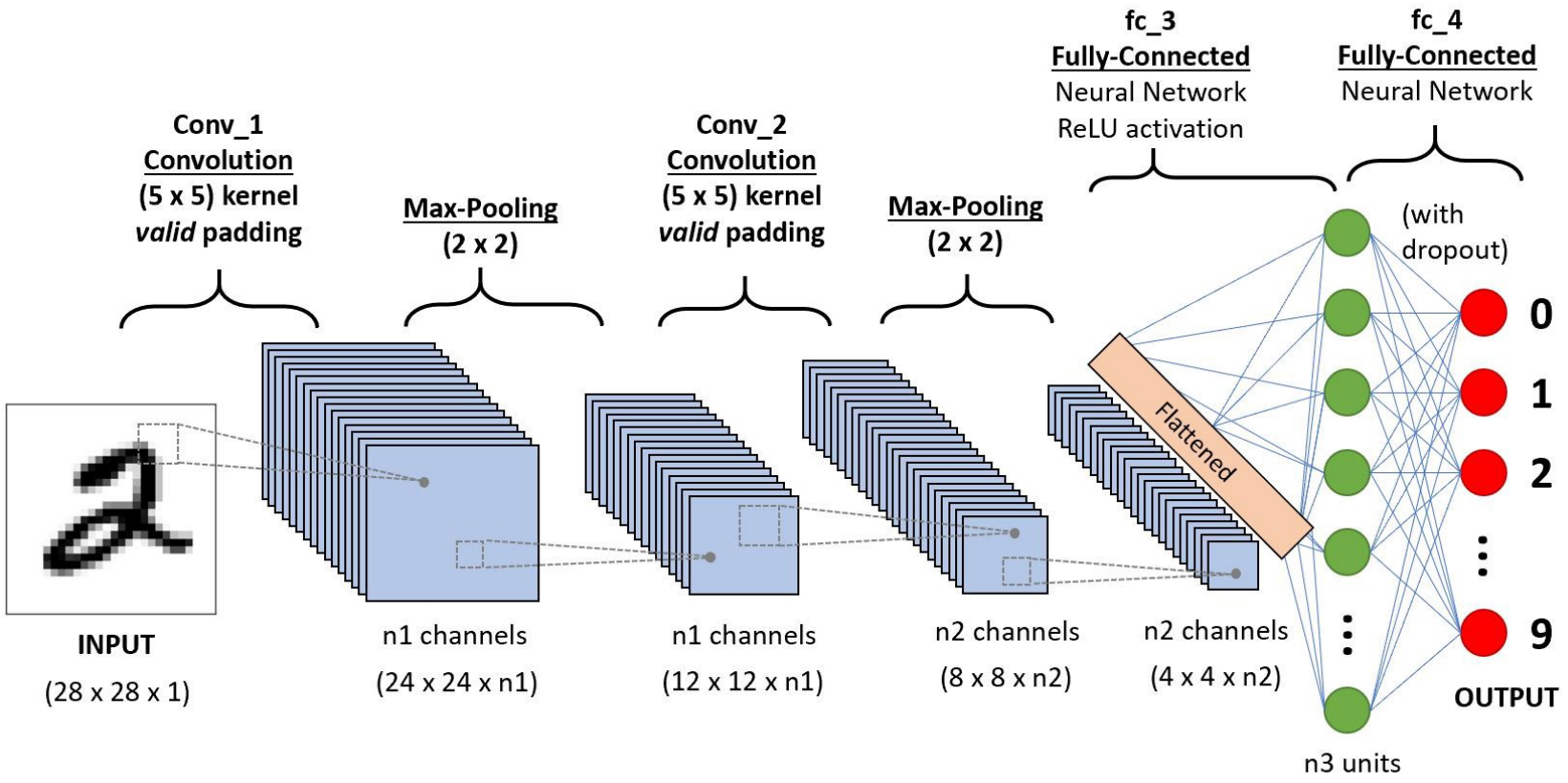


# Convolutional neural network

- Max pooling vs Average pooling



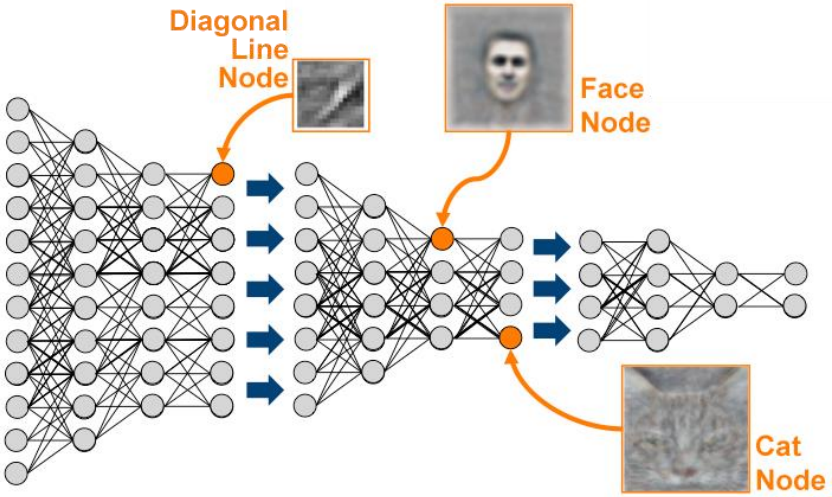
# Convolutional neural network



# Learn high level features of a cat

*“Best neuron” activation heat map*

- Training: 16.000 CPU during 3 days
- Learned high levels features of cats, human faces by watching Youtube videos
- Totally unsupervised : unlabeled data



# High level feature learning

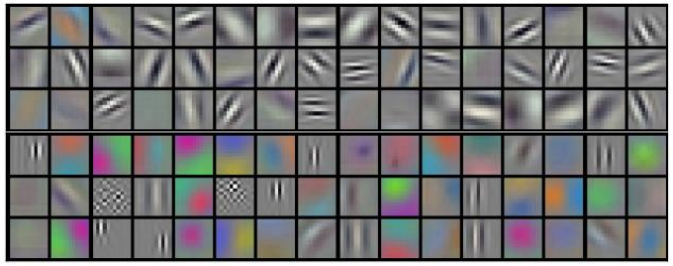
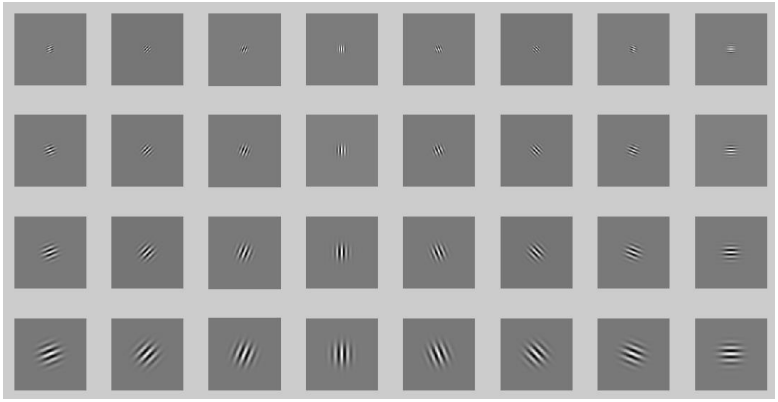


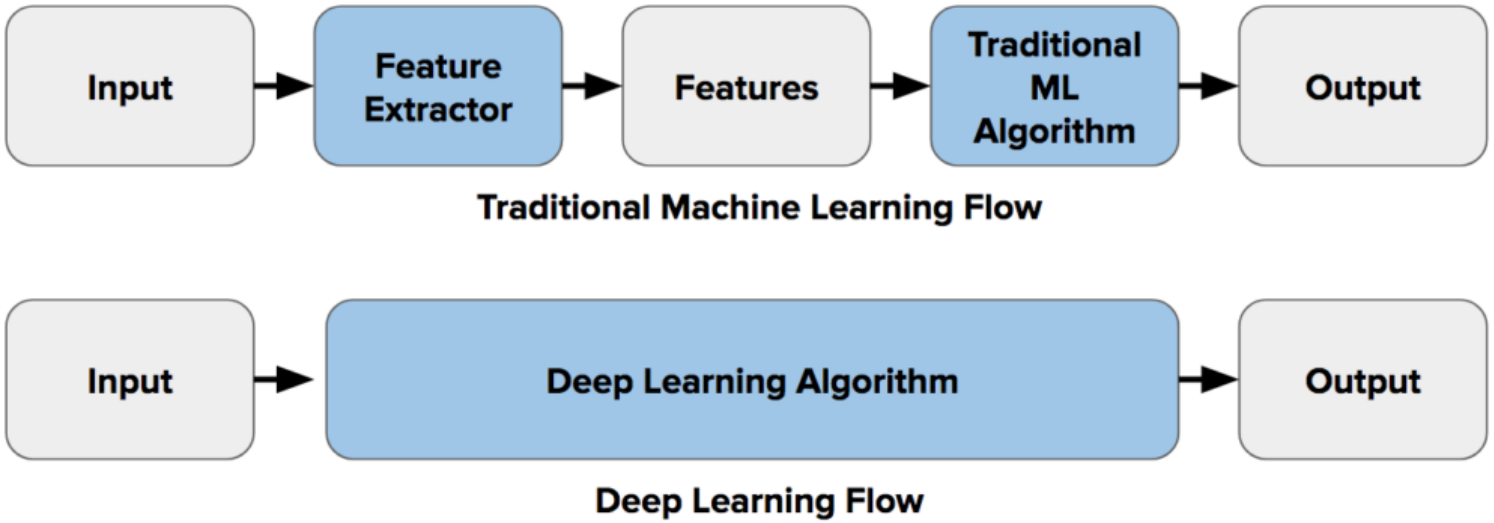
Figure 3: 96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2. See Section 6.1 for details.

The model learns some edge detection filter. We find a similar process in the cells of the primary visual cortex of the human brain

**Edge detectors filters :**



# Traditional ML vs Deep learning



# Outline

- Introduction
- Neural network
- Convolutional neural network
- **Practice with Google Colab**

# Practice with Google Colab

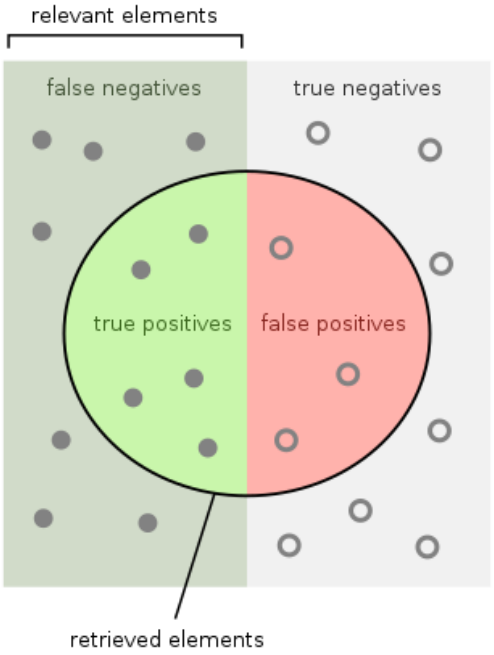
- Hand-written digits recognition with CNN
  - https://colab.research.google.com/drive/1UA-wUXx2QziKgZXTDLVo1gn0xImlwj2H

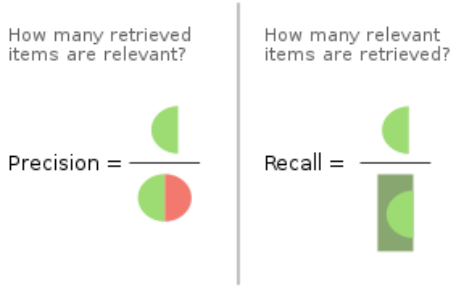
# Model evaluation

- Measurements
  - Precision, recall
  - Accuracy
  - F1-score
- Depending on problems, we need some suitable measures

# Model evaluation

- $\text{Precision} = \frac{\text{True positive}}{\text{Predicted positive}}$
- $\text{Recall} = \frac{\text{True positive}}{\text{Positive}}$
- $\text{F1-score} = \frac{2 \text{ x Precision x Recall}}{\text{Precision} + \text{Recall}}$
- $\text{Accuracy} = \frac{\text{True positive} + \text{True negative}}{\text{Positive} + \text{Negative}}$




Source: Wikipedia

# Confusion matrix

| | | Predicted | | | | |
|---|---|---|---|---|---|---|
| | | Cat | Dog | Tiger | Wolf | Total |
| Actual | Cat | **6** | 0 | 3 | 1 | 10 |
| | Dog | 2 | **4** | 0 | 4 | 10 |
| | Tiger | 3 | 3 | **3** | 0 | 9 |
| | Wolf | 1 | 4 | 1 | **2** | 8 |
| | Total | 12 | 11 | 7 | 7 | |

**>>> sklearn.metrics.classification_report**

>>> y_true = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, …]

>>> y_pred = [0, 0, 0, 0, 0, 0, 2, 2, 2, 3, …]

>>> target_names = ['Cat', 'Dog', 'Tiger', 'Wolf']

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Cat | 0.500 | 0.600 | 0.545 | 10 |
| Dog | 0.364 | 0.400 | 0.381 | 10 |
| Tiger | 0.429 | 0.333 | 0.375 | 9 |
| Wolf | 0.286 | 0.250 | 0.267 | 8 |
| accuracy | | | 0.405 | 37 |
| macro avg | 0.394 | 0.396 | 0.392 | 37 |
| weighted avg | 0.399 | 0.405 | 0.399 | 37 |

# Thank you for your attention